MODULE 7

生成式 AI 基礎：從 Transformer 到 LLM

M7 — Foundations of Generative AI

這一週不碰工具，只弄懂原理。搞清楚 LLM 到底在做什麼，
下三週在 Dify 裡設定的每一個欄位，你才知道自己在設什麼。

資料科學　2027 Spring　·　第 12 週　·　Phase C　|　謝楠楨

本講義大綱

01 技術演進與應用領域 • 從 2017 到 2026 的三個轉折

02 生成式 vs 判別式 • 跟前面模組學的差在哪

03 四種生成式架構 • GAN • VAE • Diffusion • Transformer

04 Token 與 Tokenization • 模型眼中的文字

05 Attention 機制 • 一句話說完它在做什麼

06 三種 Transformer 架構 • 為什麼 RAG 要用兩種模型

07 Embedding 與語意空間 • 文字怎麼變成有距離的數字

08 預訓練／微調／推論 • 三件常被混為一談的事

09 讓模型懂你的領域 • 四種辦法與決策順序

10 推論參數 • Temperature 、 Top-p 與其他

11 幻覺的成因 • 為什麼它不是 bug

12 模型能力與規模 • 參數量、上下文、多模態

13 推理模型與多模態 • 2025 之後的進展

14 部署選項 • 雲端 vs Ollama 本地

15 Hugging Face 生態 • 模型從哪裡來

16 AI Studio 生成式範例 • 課堂實作

17 提示詞基礎與上下文工程 • 為什麼「怎麼問」差這麼多

18 倫理與資料落地 • 醫療場域的界線

這一週在整個課程的位置

你已經走過的路（M1–M6）

- 資料工程 → 監督式學習（迴歸、分類）→ 非監督式學習（分群、降維）
- 共同點：多半是「給我資料，我告訴你一個答案」——分類與迴歸屬判別式模型
- 評估方式清楚：準確率、 RMSE 、 Silhouette 都是可以算出來的數字

這一週要換一個世界觀（M7）

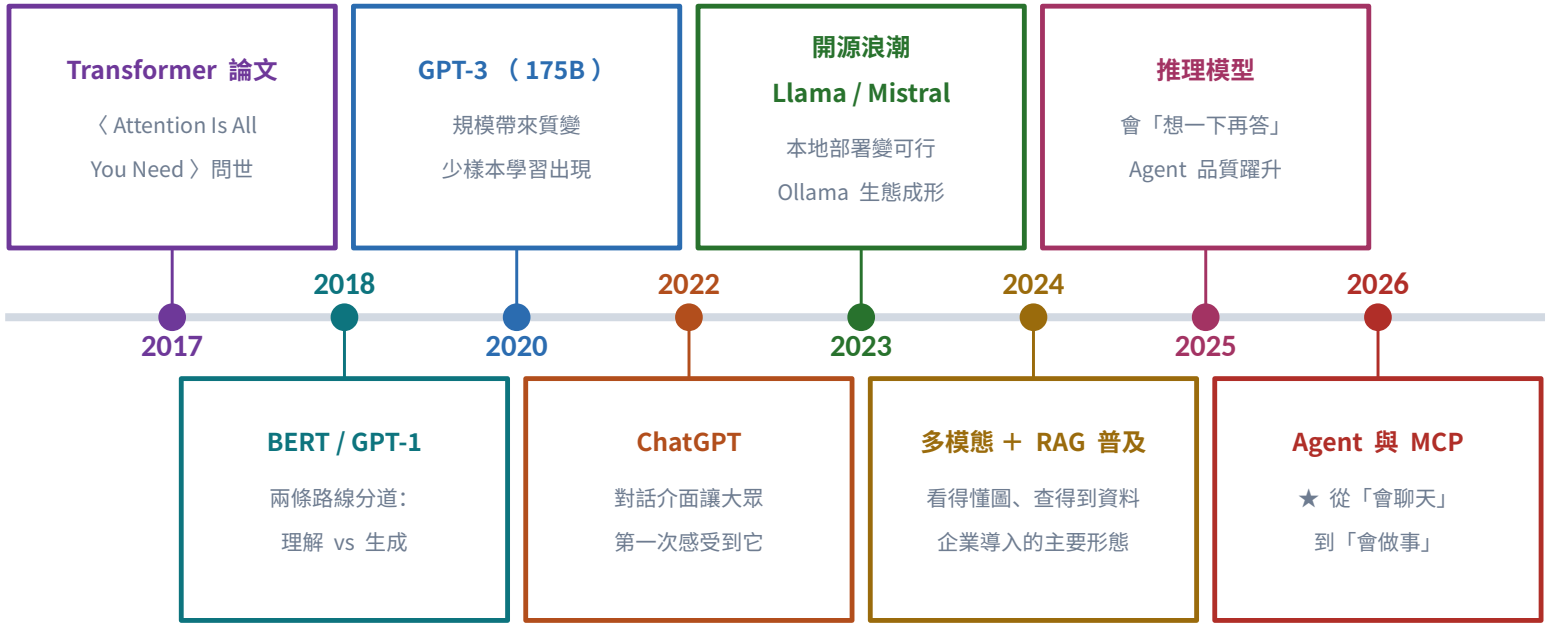
- 生成式模型不回答「是哪一類」，它「產生一個新的東西」
- ★ 這代表評估方式、失敗模式、風險型態全都不一樣
- 「模型準確率 92% 」在這裡幾乎沒有意義

為什麼要先學原理再學工具（M8）

- M8 在 Dify 裡每一個欄位—— Top-K 、 Temperature 、上下文長度、模型選擇
- 背後都是這一週的概念。不懂原理，你只能照抄別人的設定值
- ★ 出事的時候（幻覺、成本爆掉、答非所問）你會不知道從哪查起

本模組不做工具操作，唯一的實作是 AI Studio 的生成式範例與 Ollama 本地部署。

技術演進：從一篇論文到會做事的 Agent



★ 這張圖有兩件事值得注意

① 從 2017 到 ChatGPT 隔了五年 —— 技術突破與大眾感知之間有很長的落差。

② 2023 年之後每一年的主題都在往「能做事」移動，這正是 M8 期末專題要做的方向。

2017 是真正的起點

〈Attention Is All You Need〉。這篇論文之後的所有發展，
都建立在同一個機制上。

★ 注意那五年的落差

2017 技術突破 → 2022 ChatGPT 才讓大眾感受到。
技術成熟與應用普及之間，永遠有一段時間差。

2023 之後的主題一直在移動

開源本地化 → 多模態與 RAG → 推理 → Agent。
每一年都在往「能做事」而不是「會聊天」靠近。

這正是課程的終點

M8 期末專題做的就是 2026 年這一格：
一個會做事的 Agent。

★ 原生圖形，可直接在 PowerPoint 編輯

三個轉折點，各自解決了什麼問題

轉折	之前的困境	解決了什麼	留下什麼新問題
2017 Transformer	RNN 逐字讀，長文本前後文接不上；無法平行運算，訓練極慢	距離不再衰減；可大規模平行訓練	計算量隨長度平方成長 → 上下文有上限
2022 ChatGPT	模型能力已具備，但一般人接觸不到、也不知道能拿來做什麼	對話介面把門檻降到零	★ 大眾開始把它當成「知識來源」，而它其實不是
2024–26 RAG 與 Agent	模型知識過時、不知道你的資料、只會講不會做	RAG 補事實、工具調用補行動力	可靠性與可稽核性成為主要瓶頸——本課程的重心

★ 看得出一個規律：每次突破都會解決舊問題、同時製造新問題。目前這一輪的新問題，就是你們要處理的。

PART 01

生成式與判別式

Generative vs Discriminative

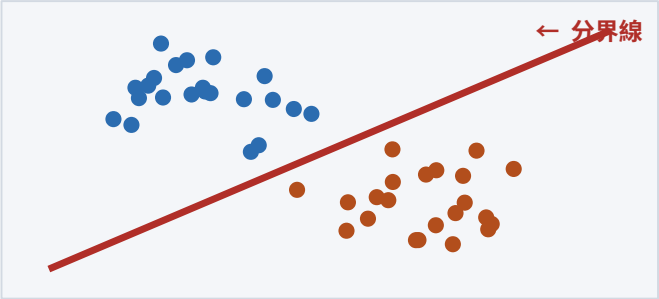
同樣叫做「模型」，但這兩類東西學的是完全不同的東西。
搞清楚這個差別，你就懂了一半。

兩類模型的根本差別

判別式模型　Discriminative

M2–M5 的分類與迴歸都是這一類

學的是「分界線」



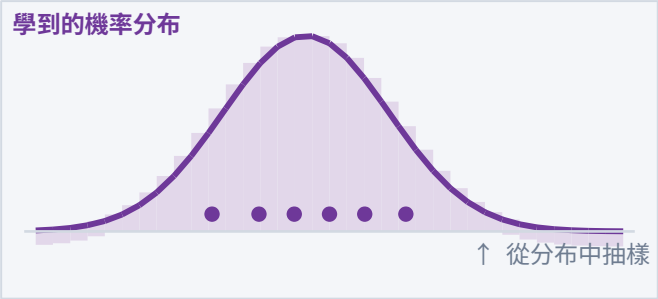
P(y | x)　·　給我一筆資料，我告訴你它屬於哪一類

問題形式：這位病人會不會再入院?

生成式模型　Generative

M7–M8 要學的是這一類

學的是「資料長什麼樣」



P(x)　·　學會分布之後，可以抽樣出新的資料

問題形式：幫我寫一份出院衛教說明

★ 關鍵差別：判別式回答「是哪一類」，生成式產生「一個新的東西」

這也是為什麼生成式模型會「編」——它本來就是設計來產生新內容的，產生沒見過的文字是正常運作。

評估方式也跟著不同：分類看準確率，生成看流暢度、事實正確性、有沒有幫上忙。

判別式：學分界線

P(y|x) 。給一筆資料，回答它屬於哪一類。
M2-2 的所有分類器都是這一類。

生成式：學資料分布

P(x) 。學會「資料長什麼樣」之後，
可以從分布中抽樣出新的東西。

★ 為什麼生成式會「編」

它本來就是設計來產生新內容的。
產生一段沒見過的文字，對它來說是正常運作，
不是故障。

評估方式也不同

分類看準確率；生成看流暢度、事實正確性、
有沒有幫上忙——
這些多半沒有單一數字可算。

原講義： GAI 1. Introduction to Generative AI.pptx　★ 原生圖形，可直接在 PowerPoint 編輯

具體對照：同一個醫療場景，兩類模型各自能做什麼

情境	判別式模型（M2–M5）	生成式模型（M7–M8）
再入院風險	輸入 30 個特徵 → 輸出「高風險 0.82」	不適合。這是分類問題，用邏輯斯迴歸更準也更可解釋
出院衛教說明	做不到。它只能選類別，不能寫文章	★ 依病人狀況產生一段個別化的說明文字
病歷摘要	做不到	★ 讀完長篇病程紀錄，整理成 SOAP 格式
檢驗數值異常偵測	★ 用 Isolation Forest 或統計法，穩定可稽核	不適合。數值判斷不該交給會「生成」數字的模型
衛教問答	做不到（除非做成固定 FAQ 比對）	★ 理解自然語言問題並回答
長照等級判定	可做成分類器，但需大量標註資料	可用 RAG ＋ LLM，資料需求低但要人工覆核

★ 判準：答案是「從已知選項中選一個」→ 判別式；答案是「一段沒有標準答案的內容」→ 生成式。

隨堂練習：這個醫療場景該用哪一類模型？

1 再入院風險： 30 個特徵 → 高風險 0.82

判別式

分類問題，邏輯斯迴歸更準也更可解釋

2 依病人狀況產生個別化的出院衛教說明

生成式

答案是一段沒有標準答案的內容

3 讀完長篇病程紀錄，整理成 SOAP 格式

生成式

摘要與格式轉換

4 檢驗數值異常偵測

判別式

用 Isolation Forest 或統計法，穩定可稽核；數值判斷不該交給會「生成」數字的模型

5 理解家屬的自然語言問題並回答照護問題

生成式

衛教問答

6 把通報分成五種事件類型

判別式

從已知選項中選一個

判準：答案是「從已知選項中選一個」→ 判別式（M2–M5）；答案是「一段沒有標準答案的內容」→ 生成式。很多專題兩者都要：判別式模型算風險，生成式模型把結果講成人聽得懂的話。

六大應用領域：醫護場域的具體例子

文字生成　Text

- 衛教說明生成
- 病歷摘要
- 出院指導單
- 公文與報告

文字理解　Understanding

- 分類與分級
- 關鍵資訊抽取
- 情緒與滿意度
- 檢索與問答

影像　Vision

- 醫學影像輔助判讀
- 報告掃描檔 OCR
- 衛教圖示生成
- 手寫病歷辨識

語音　Audio

- 門診對話轉文字
- 衛教語音導覽
- 長者語音互動
- 會議紀錄

程式碼　Code

- 資料清理腳本
- 分析流程自動化
- **★ M10 Vibe Coding**
- 儀表板產生

多步任務　Agent

- 個案評估流程
- 資料查核與通報
- **★ M8 期末專題**
- 跨系統整合

★ 你的機會多半不在「做模型」，而在「知道哪個場景值得做」

以及做出來能不能用 —— 這正是護理與健康科技背景在這一波裡最不容易被取代的部分。

文字生成與理解

目前最成熟、也最容易上手的兩塊。
本課程 M8 的專題多半落在這裡。

影像與語音

醫學影像判讀屬於高度管制領域；
但報告掃描 OCR、門診對話轉文字已相當實用。

程式碼

★ M10 的 Vibe Coding。對非資工背景的你，
這可能是投報率最高的一塊。

★ 多步任務（Agent）

M8 期末專題的方向。也是 2026 年產業投入最多的地方。

★ 原生圖形，可直接在 PowerPoint 編輯

護理與健康科技領域的六個可行切入點

切入點	做什麼	難度	資料取得
衛教內容個別化	依病人年齡、識能、疾病產生客製衛教說明	低	公開衛教素材
長照給付諮詢	依個案狀況判定等級並說明可申請服務	中	★ 公開法規（本課程範例）
病歷摘要與交班	把長篇紀錄整理成 SOAP 或 ISBAR	中	需去識別化，取得較難
問卷與量表輔助	協助填答、即時說明題意、初步彙整	低	自建題庫
文獻查證助理	針對臨床問題檢索並摘要實證文獻	中	PubMed 等公開來源
照顧者支持	回答家屬的照護操作問題、提供資源指引	低	★ 公開衛教與社會資源

★ 標星號的兩項資料最容易取得，適合作為期末專題起點。避開需要真實病歷的題目。

四種生成式模型架構

GAN	VAE	Diffusion	Transformer
生成器 vs 鑑別器	編碼 → 潛在空間 → 解碼	逐步去除雜訊	注意力機制
對抗訓練：生成器 → 鑑別器 ←	潛在空間：編碼 → z → 解碼	逐步去噪：雜訊 → 清晰	注意力連線
生成器造假	壓縮到低維	純雜訊開始	同時看全部詞
鑑別器抓假	在潛在空間取樣	一步步去噪	算出彼此關聯
互相對抗變強	解碼回原空間	還原成清晰影像	預測下一個 token
影像合成、風格轉換 合成訓練資料	異常偵測、降維 科學數據生成	影像生成主流 DALL・E 、 Stable Diffusion	★ 所有 LLM 的底層 本課程重點

★ 本課程接下來全部聚焦在 Transformer

另外三種在影像與科學資料上仍是主流，但所有主流 LLM 的底層都是 Transformer 。

GAN

生成器造假、鑑別器抓假，兩者對抗。
2026 年多用於局部影像修補與合成訓練資料。

VAE

壓縮到潛在空間再解碼回來。
現在主要用在異常偵測與科學資料降維。

Diffusion

從純雜訊開始逐步去噪。
目前影像生成的主流（DALL・E 、 Stable Diffusion ）。

★ Transformer

注意力機制。所有主流 LLM 的底層架構，
本課程接下來全部聚焦在這一個。

原講義： GAI 1. Introduction to Generative AI.pptx ★ 原生圖形，可直接在 PowerPoint 編輯

PART 02

Token： 模型眼中的文字

Tokenization

模型看不到「字」，它只看到編號。

這件事同時決定了你的帳單、上下文上限，以及它為什麼數不清字數。

Tokenization 實例

句子：「病人有高血壓病史，需長期服藥。」 → 15 tokens



中文：約 0.6–1.5 個 token ／字（示意為 1）

- 上面 15 個字 → 約 15 tokens
- 同樣意思的英文約 10 tokens （依模型而異）

★ 舊 tokenizer 下中文常較貴；新一代差距縮小

英文： 1 token ≈ 0.75 個單字 ≈ 4 個字元

- "hypertension" → hyper ＋ tension
- " 病人 " 這種常見詞可能合併成 1 個 token
- 切法由該模型的 tokenizer 決定，各家不同

為什麼你必須知道 token 這件事

① 計費以 token 計 —— 輸入與輸出分開計價，輸出通常比輸入貴 3–5 倍

② 上下文長度以 token 計 —— System Prompt ＋ 知識庫段落 ＋ 對話歷史都佔額度

③ 模型看不到「字」，只看到 token 編號 —— 這是它數不清字數的根本原因

中文的粗估

約 0.6–1.5 個 token ／字，依 tokenizer 而異。
新一代 tokenizer 對中文更省。

英文的粗估

1 token ≈ 0.75 個單字 ≈ 4 個字元。
少見的長詞會被切成好幾個 token 。

切法各家不同

每個模型有自己的 tokenizer 。
同一句話在不同模型算出的 token 數不一樣。

★ 實測工具

OpenAI 的 tokenizer 頁面可以直接貼字進去看切法。
第一次看到中文被怎麼切，多數人會很意外。

原講義： GAI 1. Introduction to Generative AI.pptx pp.62 ★ 原生圖形，可直接在 PowerPoint 編輯

算一次帳：一個衛教問答機器人的月成本

用真實的 token 結構估算，數字比你想的具體

成本估算（ M8 會實際設定這些參數）

【單次問答的 token 組成】
System Prompt （四段式，含限制與格式） 約 450 tokens
知識庫召回段落（Top-K = 5，每段約 300 字） 約 1,500 tokens
對話歷史（前三輪） 約 600 tokens
使用者本次提問 約 30 tokens
———————————————————————————————
輸入合計 約 2,580 tokens
模型輸出（四段式回答） 約 400 tokens

【成本估算】（費率隨模型與時間變動，此處僅示範算法）
假設輸入 US$0.15 / 1M tokens、輸出 US$0.60 / 1M tokens
單次 = 2,580 × 0.15/1M + 400 × 0.60/1M
= 0.000387 + 0.000240 = US$0.000627 （約新臺幣 0.02 元）

每天 500 次問答 × 30 天 = 15,000 次
月成本 = 15,000 × 0.000627 ≈ US$9.4 （約新臺幣 300 元）

【看出什麼】
① 知識庫召回段落佔了輸入的 58% —— Top-K 調大，成本線性上升
② 輸出比輸入貴 4 倍，但量少，所以輸入才是成本主體
③ 這個量級的成本很低 —— 真正會爆掉的是 Agent （輪數不固定）

隨堂檢核 1 ｜生成式與 Token

Q1

為什麼「模型準確率 92%」這句話用在 LLM 上幾乎沒有意義?

準確率需要有標準答案可比對。生成式模型產生的是開放式內容，同一個問題有無數個合理答案，也有無數個看似合理但錯誤的答案。要評估得看事實正確性、格式一致性、有沒有幫上忙——多半要人工或用另一個模型來評。

Q2

同樣一句話，中文和英文哪個 token 數多？這對你有什麼影響?

多數模型中文較多（依 tokenizer 而異，新一代差距縮小）。影響有三：成本較高、同樣的上下文長度能塞的中文內容較少、估算知識庫段落能不能放得下時要用中文的比例算。

Q3

為什麼 LLM 常常數不清一句話有幾個字?

因為它根本看不到「字」。它看到的是 token 編號，一個 token 可能是半個詞也可能是兩個字。要它數字數，等於要它做一件它的輸入表示法裡沒有的事。

Q4

上面的成本估算裡，哪一項最值得優化?

知識庫召回段落（佔輸入 58%）。降低 Top-K、開啟 Rerank 讓少而精、縮短切塊長度，都能直接降成本。 System Prompt 雖然每次都送，但只佔 450 tokens，優化空間有限。

PART 03

Attention 與 Transformer

The Attention Mechanism

2017 年那篇論文的標題就是〈Attention Is All You Need〉。
整個生成式 AI 的爆發，起點就在這個機制。

Attention 在做什麼



詞 × 詞的注意力權重矩陣

	這	病	人	血	糖	他
這	1.00	0.22	0.18	0.10	0.08	0.15
病	0.22	1.00	0.90	0.30	0.26	0.85
人	0.18	0.90	1.00	0.28	0.24	0.92
血	0.10	0.30	0.28	1.00	0.88	0.35
糖	0.08	0.26	0.24	0.88	1.00	0.30
他	0.15	0.85	0.92	0.35	0.30	1.00

紅框＝「他」這一列：最深的兩格就是「病」「人」

為什麼這件事是突破

舊做法　RNN ／ LSTM

一個字一個字往後讀，讀到「他」

時前面的「病人」已經模糊了

Attention

一次看全部，直接算出「他」跟每個

詞的關聯 —— 距離多遠都不衰減

代價

要算「每個詞對每個詞」的關聯，

計算量隨長度平方成長

★ 這就是上下文長度有上限、長文特別貴的原因

問題

模型讀到「他」的時候，怎麼知道「他」指的是誰？

做法

對句子中每一個詞算一個權重。權重高＝關聯強。
圖中「病人」的權重最高（0.85 、 0.92 ）。

★ 突破在哪

舊做法（ RNN ／ LSTM ）一個字一個字往後讀，

讀到「他」時前面的資訊已經模糊。 Attention 一次看全部。

代價

要算「每個詞對每個詞」的關聯，計算量隨長度平方成長。
這就是上下文長度有上限、且長文特別貴的原因。

原講義： GAI 1. Introduction to Generative AI.pptx　pp.65　★ 原生圖形，可直接在 PowerPoint 編輯

為什麼 Attention 解決了醫療文本的核心難題

醫療文本的特性

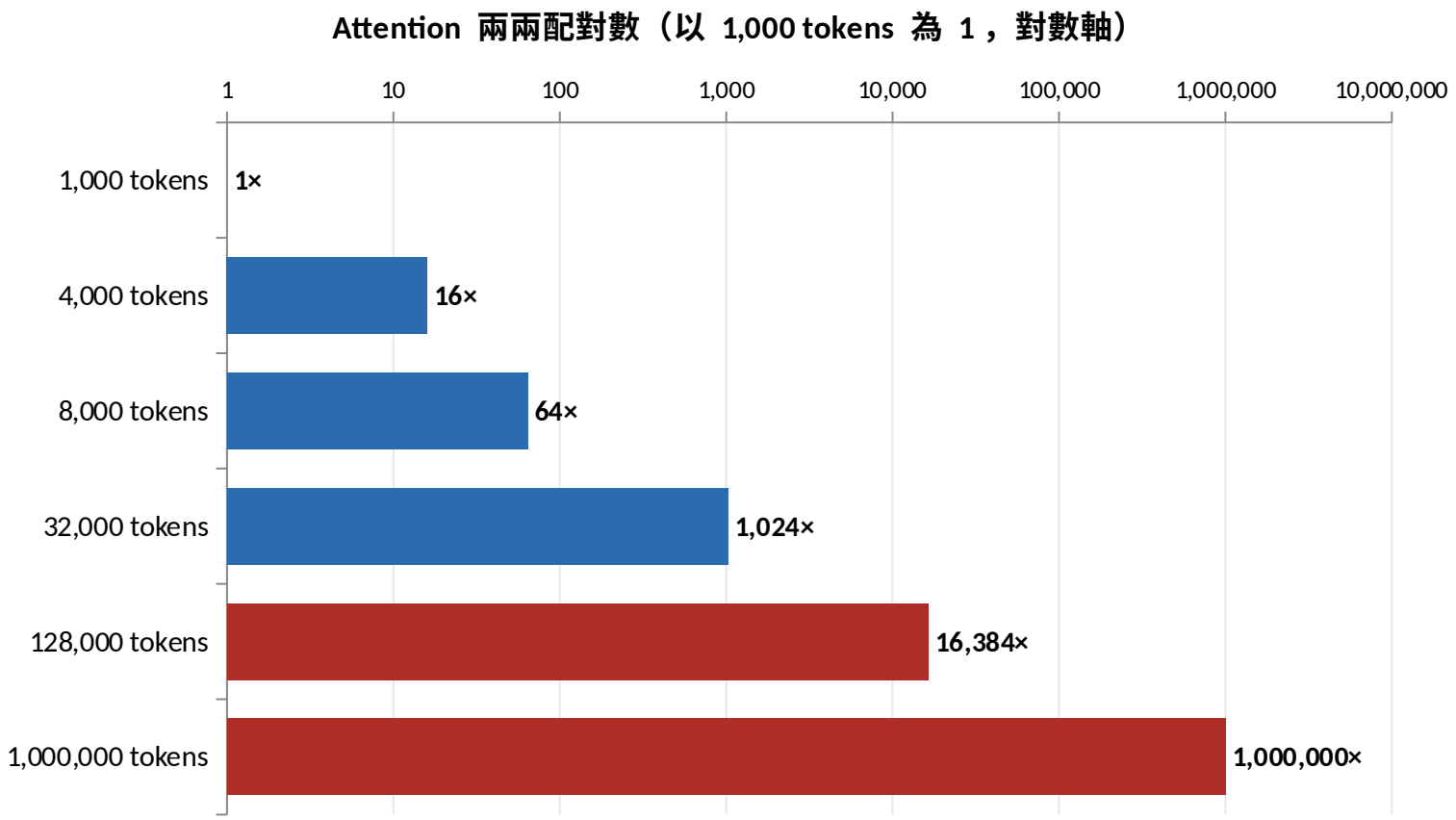
- 長距離指涉：「該病人」「上述症狀」「此藥物」常常指涉很前面的內容
- 巢狀修飾：「因高血壓合併糖尿病而長期服用之降血壓藥物」
- 縮寫與同義詞： HTN ／高血壓／ hypertension 指同一件事
- 否定與條件：「無胸痛」「若出現發燒則…」的語意要能正確捕捉

Attention 怎麼處理

- 長距離指涉 → 距離不影響權重計算，多遠都能連上
- 巢狀修飾 → 多頭注意力可同時追蹤語法與語意等多層關係
- 縮寫同義 → 在 embedding 空間中位置相近，自然被連在一起
- ★ 但否定詞仍是弱點：「無胸痛」被誤讀成「胸痛」是實務常見錯誤

★ 最後一點很重要：做醫療 NLP 時，否定偵測（ negation detection ）通常要另外處理，不能全交給模型。

實例：為什麼上下文有上限——計算量隨長度平方成長



每個 token 都要看其他所有 token

n 個 token 就有 n × n 組配對。長度變 2 倍，計算量變 4 倍；變 10 倍，計算量變 100 倍。

128K 是 1K 的 16,384 倍

一百萬 tokens 是 1K 的一百萬倍。所以長上下文的模型更貴、更慢，廠商也要用各種近似技巧。

對你的意義

上下文不是越長越好：塞太多段落，貴、慢，而且重點會被稀釋（第 61 張的上下文工程）。

這是原始 Attention 的量級直覺；實際產品用 FlashAttention、稀疏注意力等方法降低成本，但「越長越貴」的方向不變。

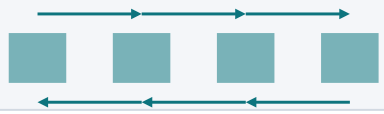
三種 Transformer 架構

Encoder-only

BERT 系列

雙向：同時看左右文

左右都看得到



- 填空（克漏字）訓練
- 擅長「理解」
- 分類、 NER 、檢索

★ RAG 的嵌入模型

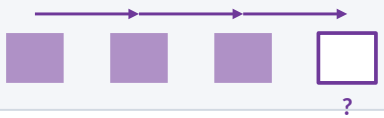
就是這一類

Decoder-only

GPT 系列・所有主流 LLM

單向：只能看左邊

只能看左邊 → 猜下一個



- 預測下一個 token
- 擅長「生成」
- 對話、寫作、程式

★ 你在 Dify 裡

呼叫的就是這一類

Encoder-Decoder

T5 、 BART

先讀完，再生成

讀完 → 交給解碼端



翻譯、摘要等固定轉換

- 序列轉序列訓練
- 擅長「轉換」
- 翻譯、摘要

AI Studio 的

T5 範例屬於這類

★ 為什麼 RAG 要用兩種模型

嵌入用 Encoder-only （把文字變向量）、生成用 Decoder-only （產生答案）—— 這是兩件不同的事。

Encoder-only （BERT）

雙向，同時看左右文。擅長「理解」。
★ RAG 的嵌入模型就是這一類。

Decoder-only （GPT）

單向，只能看左邊，預測下一個 token 。擅長「生成」。
★ 你在 Dify 呼叫的就是這一類。

Encoder-Decoder （T5）

先讀完再生成。擅長「轉換」：翻譯、摘要。

AI Studio 的 T5 範例屬於這類。

★ 為什麼 RAG 要兩種模型

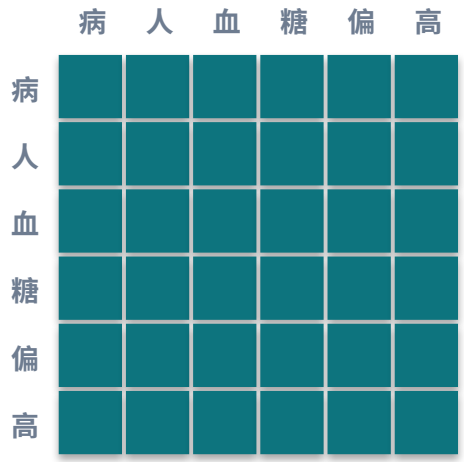
嵌入用 Encoder-only （把文字變向量），
生成用 Decoder-only （產生答案）。這是兩件不同的事。

原講義： GAI 1. Introduction to Generative AI.pptx pp.98 ★ 原生圖形，可直接在 PowerPoint 編輯

實例：誰看得到誰——三種架構的注意力遮罩

Encoder-only （BERT 系）

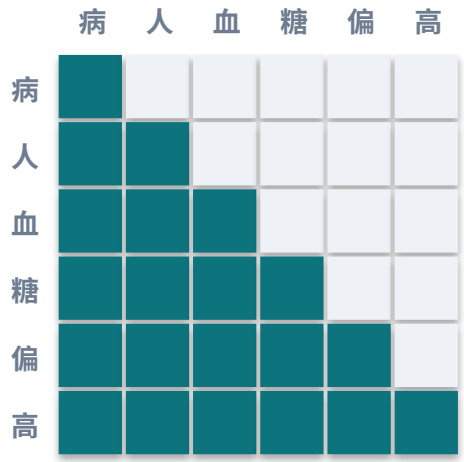
每格＝這一列的字看得到那一欄



每個字都看得到全部（雙向）。用克漏字訓練，擅長「理解」：分類、 NER 、檢索。★ RAG 的嵌入模型就是這一類。

Decoder-only （GPT 系）

只能往左看



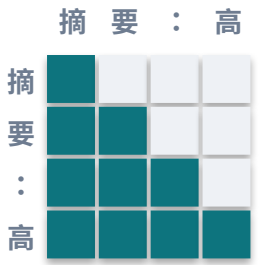
只能看自己和左邊（單向）。訓練目標是預測下一個 token ，擅長「生成」。★ 你在 Dify 呼叫的就是這一類。

Encoder-Decoder （T5 、 BART ）

生成時看原文



看自己左邊



先雙向讀完原文，再一個字一個字生成（看原文＋自己左邊）。擅長「轉換」：翻譯、摘要。

有色＝看得到、灰＝被遮住。同樣是 Transformer ，差別只在這張遮罩：雙向適合讀懂，單向才能一個字一個字往下生成。

三種架構的實際用途對照

架構	代表模型	訓練方式	在你的專題裡的角色
Encoder-only	BERT 、 RoBERTa 、 bge / text-embedding 系列	克漏字（遮住一個詞讓它猜）	★ 知識庫的嵌入模型：把段落與問題轉成向量
Decoder-only	GPT 、 Claude 、 Gemini 、 Llama 、 Qwen 、 DeepSeek	預測下一個 token	★ 主力：生成回答、判斷分類、 Agent 的大腦
Encoder-Decoder	T5 、 BART 、 mT5	把一段序列轉成另一段序列	翻譯、摘要等固定轉換任務； AI Studio 範例用 T5

★ 在 Dify 的知識庫設定裡，「嵌入模型」與「 LLM 」是兩個分開的下拉選單——原因就在這張表。

隨堂練習：這件事交給哪一種架構?

1 把知識庫段落與使用者問題轉成向量

Encoder-only

嵌入模型

2 依召回的段落產生回答

Decoder-only

生成

3 把英文衛教單翻成中文

Encoder-Decoder

序列轉換

4 Agent 規劃下一步要呼叫哪個工具

Decoder-only

Agent 的大腦

5 判斷一則留言的情緒類別

Encoder-only

理解型任務（也可用 Decoder 做）

6 把長篇病程紀錄濃縮成三句摘要

Encoder-Decoder

經典的摘要任務；現在也常直接交給 Decoder-only 的 LLM

一個 RAG 系統通常同時用到兩種： Encoder-only 的嵌入模型負責「找」， Decoder-only 的 LLM 負責「寫」。 M8 在 Dify 裡會分別設定這兩個模型。

PART 04

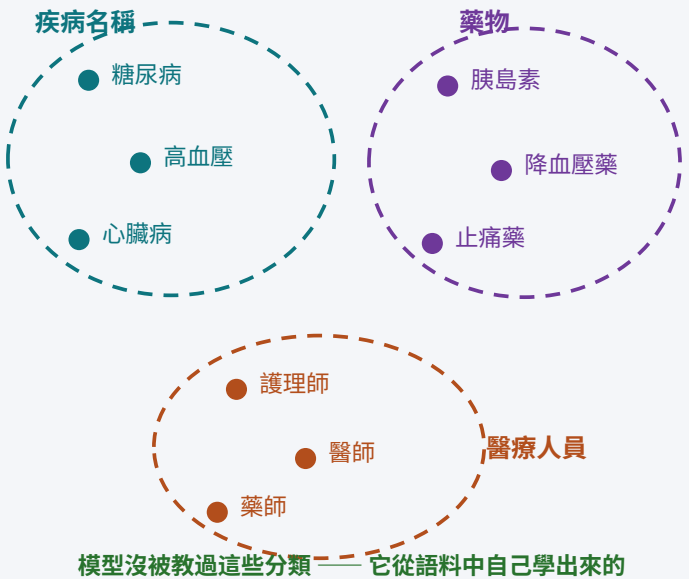
Embedding：語意的數學表示

Embeddings & Semantic Space

文字沒辦法直接計算。要讓電腦判斷「兩句話意思像不像」，
得先把文字變成有「距離」的數字。

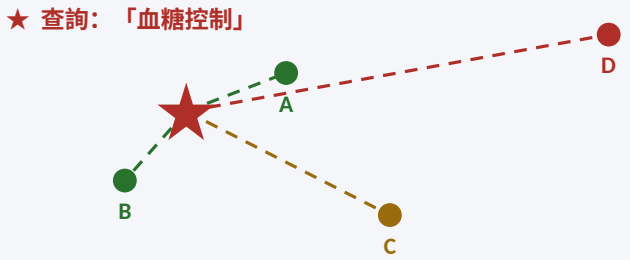
Embedding： 把文字變成有距離的數字

左圖　語意相近的詞，向量位置也相近



模型沒被教過這些分類——它從語料中自己學出來的

右圖　RAG 的召回：找出離查詢最近的段落



A	糖尿病照護指引	距離 0.28	✔
B	血糖監測方法	距離 0.35	✔
C	高血壓用藥	距離 0.98	✘
D	長照給付標準	距離 2.87	✘

距離最近的兩段被召回，餵給 LLM 當作答依據

★ 這就是 M8 「知識檢索節點」的全部原理

把問題與所有段落都轉成向量，再挑距離最近的幾段餵給 LLM —— 所以查詢「血糖控制」召得回「糖尿病照護指引」。

實務上是幾百到幾千維；圖上畫成二維只是為了看得懂。

左圖　語意群聚

疾病名稱聚在一起、藥物聚在一起、醫療人員聚在一起。
模型沒被教過這些分類——它從語料中自己學出來的。

右圖　RAG 的召回

把問題也轉成向量，算出離它最近的段落。
★ 這就是 M8 「知識檢索節點」的全部原理。

為什麼是「語意」不是「關鍵字」

查詢「血糖控制」能召回「糖尿病照護指引」，

即使那份文件裡沒有出現「血糖控制」四個字。

維度

實務上是幾百到幾千維。圖上畫成二維只是為了看得懂。

實際算一次：餘弦相似度

RAG 用來衡量「像不像」的計算，本質上就這麼簡單

手算範例（數值已驗算）

【簡化示範】把每個句子表示成 4 維向量（實務是 768 或 1536 維）

```
查詢 Q        「病人的血糖偏高」     → [0.8, 0.6, 0.1, 0.0]
段落 A        「糖尿病的血糖監測方式」  → [0.7, 0.7, 0.1, 0.1]
段落 B        「長照給付額度標準」    → [0.1, 0.0, 0.9, 0.4]
```

【餘弦相似度】 cos(Q,A) = (Q · A) / (|Q| × |A|)

```
Q · A  = 0.8×0.7 + 0.6×0.7 + 0.1×0.1 + 0.0×0.1 = 0.56+0.42+0.01+0 = 0.99
|Q|    = sqrt(0.64+0.36+0.01+0)    = sqrt(1.01) = 1.005
|A|    = sqrt(0.49+0.49+0.01+0.01) = sqrt(1.00) = 1.000
cos(Q,A) = 0.99 / (1.005 × 1.000) = 0.985      ← 很相似✔️

Q · B  = 0.8×0.1 + 0.6×0.0 + 0.1×0.9 + 0.0×0.4 = 0.08+0+0.09+0 = 0.17
|B|    = sqrt(0.01+0+0.81+0.16) = sqrt(0.98) = 0.990
cos(Q,B) = 0.17 / (1.005 × 0.990) = 0.171      ← 不相似✖️
```

【對應到 Dify】知識檢索節點的「Score 閾值 0.5」，比的就是這個數字。
0.985 > 0.5 → 段落 A 被召回；0.171 < 0.5 → 段落 B 被過濾掉。

Embedding 的三個實務陷阱

① 中文的嵌入模型要選對

- 用英文語料訓練的嵌入模型處理中文，效果會明顯變差
- 選擇時看模型卡是否標明支援中文（如 bge-m3 、 multilingual-e5 ）
- ★ Dify 的知識庫設定裡換嵌入模型，整個知識庫必須重新索引

② 嵌入模型換了，舊向量就不能用

- 不同模型產生的向量在不同的空間裡，彼此不可比較
- 實務上這代表：換模型＝重跑一次全部文件的嵌入，要重新計費與等待

③ 語意相似 ≠ 答案正確

- 召回的是「跟問題像的段落」，不保證那段話裡有答案
- 例：問「第 4 級給付多少」，可能召回一段在談「第 4 級的評估標準」
- ★ 這是 M8 「召回對了但答案不對」那個失效模式的根源

PART 05

預訓練、微調、推論

Pretraining / Fine-tuning / Inference

這三件事常被混為一談，但成本相差十個數量級。
分清楚它們，你才知道自己在做的是哪一件。

三個階段的成本與角色



三者的成本量級（對數尺度）—— 相差約十個數量級



★ M8 的 Dify 只做第 ③ 階段 —— 它不訓練也不微調

能換模型、能改提示詞、能接知識庫，但改不了模型本身。這一句話決定了 Dify 能做與不能做什麼。

① 預訓練

兆級 token、上萬張 GPU、數千萬美元。
只有大公司做得起，你不會做。

② 微調

領域資料數千～數萬筆，數百到數萬美元。
有專屬資料與明確需求時才做。

③ 推論

★ 你天天在做的就是這件事。呼叫模型、拿回答案，
權重完全沒有變動。按 token 計費。

★ 對 M8 的意義

Dify 只做第 ③ 階段。它不訓練也不微調——
這一句話決定了它能做與不能做什麼。

原講義： GAI 1. Introduction to Generative AI.pptx　pp.75–77　★原生圖形，可直接在 PowerPoint 編輯

微調的五種方式（原講義整理，補上實務判斷）

方式	做法	需要什麼資料	什麼時候用
監督式微調 **Supervised Fine-tuning**	給帶標籤的輸入輸出對，引導模型學特定任務	數千～數萬筆「問題 → 標準答案」	任務明確、輸出格式固定
人類回饋強化學習 **RLHF**	模型產生多個答案，由人挑最好的，再據以優化	大量人工偏好標註	要讓回答更符合人類偏好（主觀任務）
自監督學習 **Self-Supervised**	從無標籤資料自動產生標籤（如遮字讓它猜）	大量無標註語料	預訓練的主要方法； BERT 的 MLM 、 GPT 的自回歸
自適應微調 **Adaptive Fine-tuning**	用領域語料做領域適配，不需人工標註	領域文本（如醫療語料庫）	★ 想讓模型熟悉醫療用語與行文
自學習 **Self-Training**	模型用自己的預測當標籤，再訓練自己	少量標註 ＋ 大量無標註	標註昂貴或稀缺時

★ 這五種你在課程中都不會做。列出來是為了讓你聽得懂業界的討論，並知道「微調」不是單一件事。

要讓模型「懂我的領域」，有四種辦法

做法	時間	成本	知識更新	適合處理什麼
提示詞工程 Prompt	分鐘	幾乎為零	改一行字就好	行為與格式的調整
RAG 知識庫 Retrieval	小時	低（存向量庫）	換文件即可	★ 知識會變動的場景
微調 Fine-tuning	天～週	中～高	要重新訓練	風格、術語、格式的深度內化
重新預訓練 Pretraining	月	極高	不可能	不用考慮

★ 決策順序：先試提示詞 → 不夠加 RAG → 還不夠才考慮微調

多數人一開始就想微調，那幾乎總是錯的。成本、時間、技術門檻由上往下每一階都跳一個量級。

知識會變動的場景選 RAG；只有「風格與格式要內化、量大到 Prompt 塞不下」才考慮微調。

由便宜到昂貴

提示詞 → RAG → 微調 → 重新預訓練。
成本、時間、技術門檻依序遞增。

★ 決策順序

先試提示詞 → 不夠就加 RAG → 還不夠才考慮微調。
多數人一開始就想微調，那幾乎總是錯的。

知識會變動 → 選 RAG

法規、給付標準、臨床指引每年都改。
微調過的知識鎖在權重裡，要更新只能重訓。

風格要內化 → 才考慮微調

希望模型「天生」就用院內的病歷書寫格式、術語與語氣，且量大到 Prompt 塞不下時。

★ 原生圖形，可直接在 PowerPoint 編輯

RAG 與微調：一張表講清楚差別

面向	RAG	微調
解決什麼	模型「不知道」某些事實	模型「不會用」某種風格或格式
知識存在哪	外部知識庫（可隨時替換）	模型權重裡（動不了）
更新一份法規	替換文件，重新索引，幾分鐘	重新訓練，數小時～數天
可以標註來源嗎	★ 可以，能回傳召回的段落	不行，說不出答案從哪來
成本	低（存向量庫 + 每次多送 token ）	中～高（訓練 + 存放多個模型版本）
醫療場域適用度	★★ 高：可稽核、可更新、可標來源	低：無法追溯，法規更新困難
兩者可以並用嗎	★ 可以，也常這麼做：微調負責格式， RAG 負責事實	

★ 醫療場域幾乎總是先選 RAG 。「說得出答案從哪來」在臨床與稽核上是硬需求。

PART 06

推論參數與幻覺

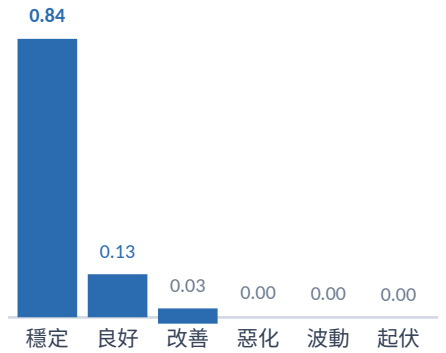
Inference Parameters & Hallucination

同一個模型、同一個問題，設定不同答案就不同。
以及那個所有人都在問的問題：它為什麼會胡說八道?

Temperature： 控制隨機性

句子：「這位病人的病情…」　　下一個詞的候選：穩定／良好／改善／惡化／波動／起伏

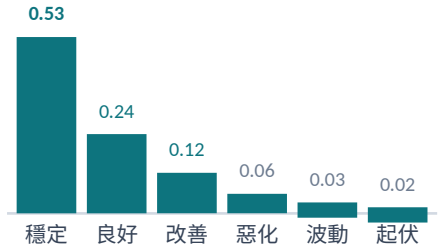
低溫 0.3



高度集中在「穩定」

→ 可重現、適合結構化輸出

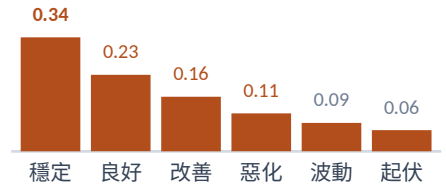
中溫 0.7



仍偏好「穩定」，但偶爾換詞

→ 一般對話的預設值

高溫 1.4



分布被拉平，什麼都可能選

→ 創意寫作；醫療場域不要用

★ 醫療應用一律用低溫（0–0.3）

Temperature 調低能減少「用詞飄移」，但擋不住「內容編造」—— 那要靠 RAG。

它只影響「從候選中怎麼挑」，不影響「候選本身對不對」。

設 0 只是讓它每次都挑同一個最高機率的答案；那個答案本來就錯的話，它會穩定地錯下去。

它在調什麼

下一個 token 的機率分布。溫度低 → 分布尖銳、幾乎總選最高機率的；
溫度高 → 分布拉平、什麼都可能被選中。

低溫 0–0.3

可重現、結構化。★ 醫療應用一律用這一段。

中溫 0.7

一般對話的預設值。有變化但不至於離譜。

★ 重要的限制

Temperature 調低能減少「用詞飄移」，但擋不住「內容編造」——那要靠 RAG。

原講義： GAI 1. Introduction to Generative AI.pptx　pp.63　★ 原生圖形，可直接在 PowerPoint 編輯

其他常見推論參數

參數	作用	醫療應用建議值	調錯會怎樣
Temperature	控制隨機性	0 – 0.3	太高 → 用詞飄移、格式不穩
Top-p （核採樣）	只從累積機率前 p 的候選中挑	0.1 – 0.5	太高 → 冷僻詞被選中的機會上升
Max tokens	限制輸出長度	依格式估算，留 20% 餘裕	太小 → 答案被硬生生截斷
Frequency penalty	抑制重複用詞	0 – 0.3	太高 → 該重複的術語被迫換詞，反而錯
Seed	固定隨機種子以求可重現	測試時固定，正式可不設	不設 → 同一問題兩次答案不同，難以除錯

★ 課堂練習：把 Temperature 設 0 與 1.2，同一個問題各跑五次，比較答案的差異程度。

幻覺是怎麼來的

LLM 實際在做的事（不是你以為的那件事）



它挑的是「最像下一個字的東西」，不是「事實上正確的東西」。

「128」機率最高就寫 128 —— 但沒有任何人量過這個數字。

① 訓練資料的空白處

模型沒學過的內容，它不會說「不知道」，而是產生一個「看起來合理」的答案

② 機率取樣的本質

每個字都是抽樣出來的。抽到低機率但錯誤的字，整句話就跟著歪掉

③ 沒有事實查核機制

模型內部沒有「這是真的嗎」這個步驟 —— 它只有「這樣接順不順」

★ 所以「幻覺」不是 bug，是這個機制的必然產物

你不可能用提示詞「關掉」它 —— 只能用 RAG 提供事實、用程式碼接管數值、用人工覆核把關。
這正是 M8 整個模組在做的事。

它在做的事

挑「最像下一個字的東西」，不是「事實上正確的東西」。

三個來源

① 訓練資料的空白處 ② 機率取樣的本質 ③ 沒有事實查核機制

★ 所以幻覺不是 bug

是這個機制的必然產物。你不可能用提示詞「關掉」它。

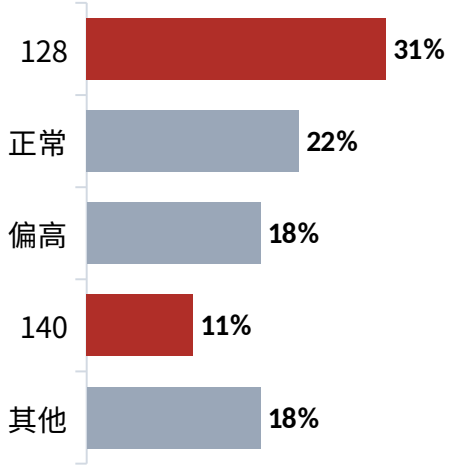
能做的三件事

用 RAG 提供事實、用程式碼接管數值、用人工覆核把關。
這正是 M8 整個模組在做的事。

★ 原生圖形，可直接在 PowerPoint 編輯

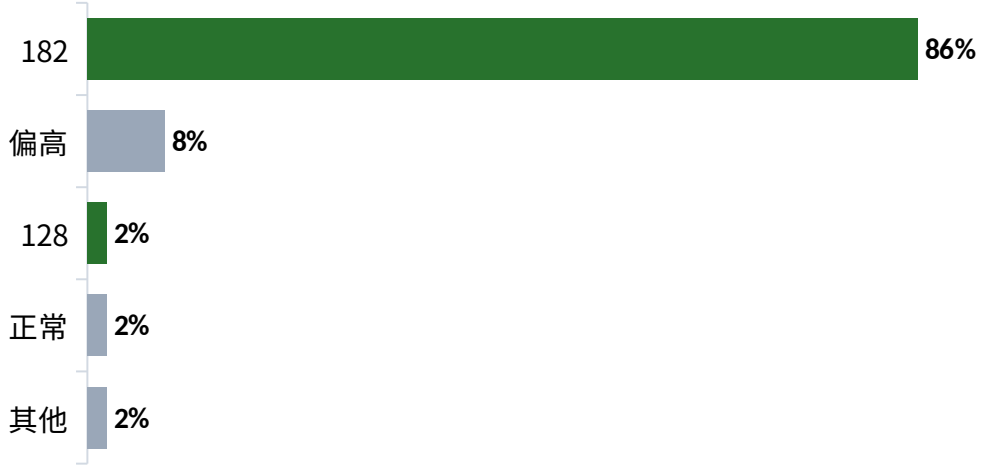
實例：「病人的血糖值為 ___ 」下一個詞怎麼挑

沒有給資料：只能「猜」一個像樣的數字



模型沒看過這位病人的檢驗結果，卻會自信地接上「 128 」——它挑的是「最像下一個詞」的東西，不是事實。 Temperature 調高，每次還可能說不同的數字。

RAG 召回檢驗報告「血糖 182 mg/dL 」



把正確資料放進上下文，機率幾乎全集中在 182 。幻覺不是 bug ，是生成的本質——對策是給它資料，並把數值交給程式碼或知識庫。

示意機率。數字類的詞（紅）在沒有資料時都是編造的。課程鐵則：所有數值一律交給程式碼或知識庫，醫療 AI 輸出一律加上「僅供參考，最終應由專業人員判斷」。

醫療場域的幻覺特別危險：三個真實的失敗樣態

為什麼比其他領域嚴重

- 錯誤的醫療資訊看起來跟正確的一模一樣——使用者無從判斷
- 劑量、頻次、禁忌症錯一個數字，後果可能不可逆
- ★ 病人與家屬正處於焦慮狀態，傾向相信任何看起來權威的答案
- 模型的語氣永遠自信，它不會說「我不太確定」

四個常見樣態

- ① 編造數值：說出一個沒人量過的檢驗值或給付金額
- ② 拼接不同來源：把 A 疾病的處置說成 B 疾病的
- ③ 過時資訊：引用已經被修訂的舊版指引或給付標準
- ④ 過度推論：從「有這些症狀」跳到「你可能得了某某病」

★ 課程鐵則：所有醫療 AI 輸出一律加上「僅供參考，最終應由專業人員判斷」，且不得提供診斷與處方。

隨堂檢核 2 ｜三階段與幻覺

Q 1

你的專題要讓 AI 熟悉最新版的長照給付標準。該微調還是用 RAG？

RAG。給付標準每年會修訂，微調把知識鎖進權重，更新只能重訓； RAG 換一份文件、重新索引就好，而且可以標註來源段落——這在稽核上是硬需求。

Q 2

Temperature 設成 0，模型就不會產生幻覺了嗎?

不會。 Temperature 只影響「從候選中怎麼挑」，不影響「候選本身對不對」。設成 0 只是讓它每次都挑同一個最高機率的答案——如果那個答案本來就是錯的，它會穩定地錯下去。

Q 3

為什麼 RAG 可以標註來源，微調不行?

RAG 的知識存在外部知識庫，回答時可以把實際召回的段落一併回傳。微調把知識揉進權重，模型自己也「不知道」某個說法是從哪一筆訓練資料學來的。

Q 4

一個病人問「我這個藥可以配酒嗎」，你的衛教機器人該怎麼設計?

這已經接近用藥諮詢，超出衛教範圍。正確設計是： System Prompt 明確限制不提供用藥建議，偵測到用藥相關問題時回覆「請洽詢藥師或主治醫師」，並提供聯絡管道。不要嘗試用 RAG 補一份藥物交互作用表就當作可以回答——責任歸屬與正確性風險都太高。

PART 07

模型規模、能力與選型

Model Scale & Capabilities

參數量、上下文長度、多模態——這些數字到底代表什麼，
以及哪些其實已經不重要了。

衡量一個模型的五個面向

面向	是什麼	實務上的意義
參數量 **Parameters**	模型內部權重的數量， 7B 、 70B 、數千億不等	大致對應能力上限，但同規模差異也很大。開源模型看這個決定跑不跑得動
訓練資料量 **Training Tokens**	預訓練看過多少 token ，兆級起跳	與參數量搭配才有意義。近年趨勢是「小模型 + 更多資料」
上下文長度 **Context Window**	單次能處理多少 token	★ 主流模型約 12.8 萬～ 100 萬 token ；但有效利用長度常低於標稱（見「中間迷失」）
多模態 **Multimodal**	能不能處理影像、音訊、影片	醫療場域關鍵：能否讀 X 光片、報告掃描檔、手寫病歷
推理能力 **Reasoning**	能不能做多步推理而不只是接話	2025 年後的主要戰場。影響 Agent 拆解任務的品質

※ 具體模型與數字變動極快（本頁整理於 2026 年 7 月）。上課時請以各家官方文件為準——這也是要教你「看哪些面向」而不是「背哪個模型最強」的原因。

選模型時真正該問的四個問題（上）

① 資料能不能出機構?

- 這是第一個問題，不是最後一個。答案是「不能」就直接排除所有雲端 API
- ★ 醫療場域：真實病歷、個案紀錄一律不得送進雲端 API
- 去識別化後仍要確認機構的資安規範是否允許

② 中文（尤其繁體與醫療用語）表現如何?

- 國際排行榜多以英文基準測試，不代表中文表現
- 實測方法：拿你自己領域的 20 個問題，各模型跑一次人工比對
- ★ 這 20 題就是你的專屬基準，比任何排行榜都有用

這兩題決定「哪些模型可以進入候選」。過不了這兩關，能力再強也不能用。

選模型時真正該問的四個問題（下）

③ 成本結構撐不撐得住預期用量？

- 用前面教的方法估算：輸入 token × 費率 ＋ 輸出 token × 費率 × 次數
- 注意 Agent 的輪數不固定，成本上限要另外設
- 免費額度用完之後的單價才是你真正要算的數字

④ 這個選擇好不好換？

- Dify 換模型只是換一個下拉選單——但 Prompt 可能要重調
- ★ 不要把系統設計綁死在單一供應商的獨有功能上
- 把 Prompt 存成獨立檔案，換模型時才不用整份重寫

★ 注意這四個問題裡，沒有一個是「哪個模型在排行榜上最強」。

推理模型： 2025 年之後的主要進展

問題：「78 歲個案，中風後右側偏癱，無法自行如廁與沐浴，配偶 75 歲。 CMS 大約第幾級？」

一般模型：直接接話

「第 5 級。」

- 答案來得很快（1 秒內）
- 成本低（只產出幾個 token）

★ 但你看不到它怎麼想的

★ 對了也不知道為什麼對，錯了也查不出哪裡錯

推理模型：先想再答

〔思考〕先看 ADL：如廁、沐浴皆需協助 → 2 項

再看 IADL 與照顧者能力 → 配偶高齡

對照量表門檻 → 落在第 5–6 級之間

偏癱屬中重度失能 → 取較高者

〔回答〕約第 6 級（示意），應由照管專員實地評估確認

差在哪、以及什麼時候值得用

- 推理模型會先產生一段「思考過程」再給答案 —— 這段思考也算 token，所以成本高、速度慢（差 5–20 倍）
- 多步推理、需要對照規則、需要權衡的任務，品質提升明顯；單純查詢事實或格式轉換則沒必要

★ 對醫療場域的價值：思考過程本身可以被檢視 —— 專業人員能看出它哪一步推錯了，這比「一個答案」有用得多

★ 但要注意：思考過程「看起來合理」不代表它真的是這樣算的

那段文字同樣是生成出來的，也不保證結論正確。不要預設「用最強的就對了」—— 那會讓專題成本失控。

一般模型

直接接話。快、便宜，但你看不到它怎麼想的。

推理模型

先產生一段思考過程再給答案。慢、貴，但過程可檢視。

★ 對醫療的價值

專業人員能看出它哪一步推錯了——
這比拿到「一個答案」有用得多。

★ 但別過度信任

思考過程「看起來合理」不代表它真的是這樣算的。
那段文字同樣是生成出來的。

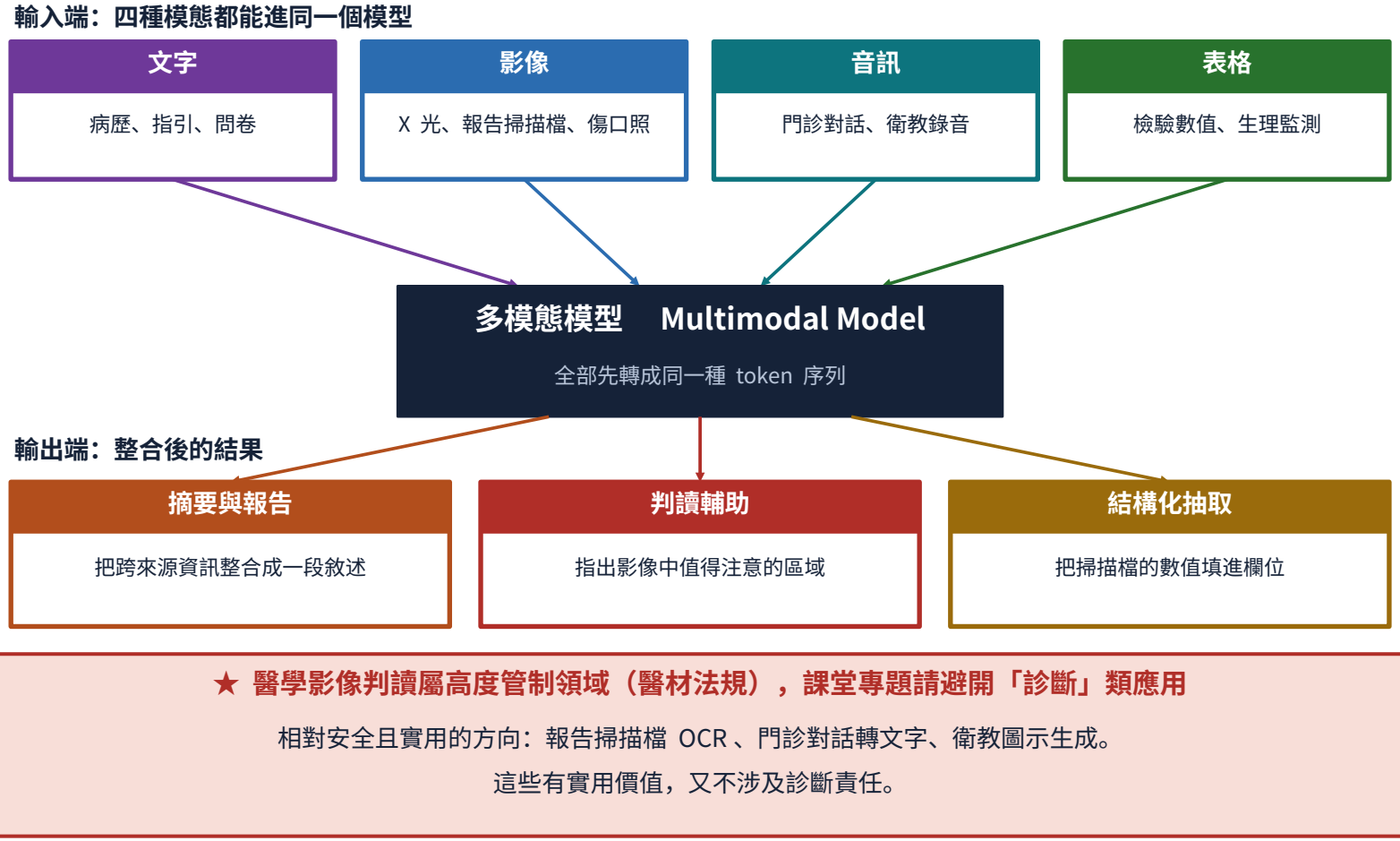
★ 原生圖形，可直接在 PowerPoint 編輯

什麼任務值得用推理模型

任務型態	值得嗎	為什麼
查詢事實（今天門診時間？）	✗ 不值得	單步查詢，推理沒有用武之地，只是白花錢
格式轉換（散文改 SOAP）	✗ 不值得	規則明確，一般模型就做得好
多步判斷（依量表判定等級）	★ 值得	要對照多個條件並權衡，思考過程可稽核
拆解模糊任務（Agent 規劃）	★ 值得	M8 的 Agent 品質很吃這個能力
寫程式與除錯	★ 值得	M10 的 Vibe Coding 明顯受益
情緒支持與陪伴對話	✗ 不值得	需要的是自然，不是推理；反而顯得生硬

★ 成本差距可能達 5–20 倍（思考過程也算 token）。不要預設「用最強的就對了」——那會讓你的專題成本失控。

多模態：不只讀文字



輸入端

文字、影像、音訊、表格都能進同一個模型。

醫護場域的實際用途

報告掃描檔 OCR、門診對話轉文字、傷口照片描述、
把掃描檔的數值填進結構化欄位。

★ 要避開的方向

醫學影像判讀屬高度管制領域（醫材法規）。

課堂專題請避開「診斷」類應用。

相對安全的切入點

OCR、語音轉文字、衛教圖示生成——
這些有實用價值又不涉及診斷責任。

★ 原生圖形，可直接在 PowerPoint 編輯

PART 08

模型跑在哪裡

Deployment: Cloud vs Local

對醫療與長照場域來說，這一節可能是整個模組最重要的部分。
因為它決定的不是效能，是資料會不會離開你的機構。

三種部署方式的取捨



★ 課程鐵則：真實病歷、個案紀錄、諮商記錄一律不得送進雲端 API

要練習就用去識別化或公開資料集。

選模型的第一個問題永遠是「資料能不能出機構」，不是「哪個模型最強」。

雲端 API

能力最強、上手最快、不用硬體。
★ 但資料送出機構外——這是最關鍵的一欄。

本地部署（Ollama）

能力中等，但資料完全不出機房。
16GB RAM 起跑得動 7B–8B 模型。

私有雲／地端企業版

能力強、合約保障資料不外流，
但成本高、要採購流程，是醫院正式導入的路徑。

★ 課程鐵則

真實病歷、個案紀錄、諮商記錄一律不得送進雲端 API。
練習用去識別化或公開資料集。

★ 原生圖形，可直接在 PowerPoint 編輯

實作：用 Ollama 在自己電腦上跑一個 LLM

十分鐘可完成。這是本模組唯一必做的環境建置

Ollama 本地部署（原講義 pp.60）

```
# ① 安裝：到 https://ollama.com 下載對應作業系統的安裝檔
#    Windows / macOS / Linux 都有；安裝後 ollama 指令即可用

# ② 下載並執行一個模型（數 GB；型號更新快，以 ollama.com/library 為準）
ollama run llama3.1:8b            # 通用，約 4.7 GB
ollama run qwen2.5:7b             # 中文表現較佳，約 4.4 GB
ollama run gemma2:2b              # 最輕量，筆電也跑得動，約 1.6 GB

# ③ 直接對話（在終端機裡）
>>> 請用三點說明高血壓的飲食注意事項

# ④ 用 API 呼叫（Dify 本地部署時就是接這個）
curl http://localhost:11434/api/generate -d '{
  "model": "qwen2.5:7b",
  "prompt": "請用三點說明高血壓的飲食注意事項",
  "stream": false
}'

# ⑤ 常用指令
ollama list                       # 看已下載的模型
ollama rm gemma2:2b               # 刪除模型釋放空間
ollama ps                         # 看目前載入記憶體的模型

# 硬體參考：7B–8B 模型約需 8 GB 可用 RAM；有獨立顯卡會快很多
# 沒有顯卡也能跑，只是慢（每秒數個 token）
```

本地模型的能力落差：誠實的說明

本地 8B 模型做得好的

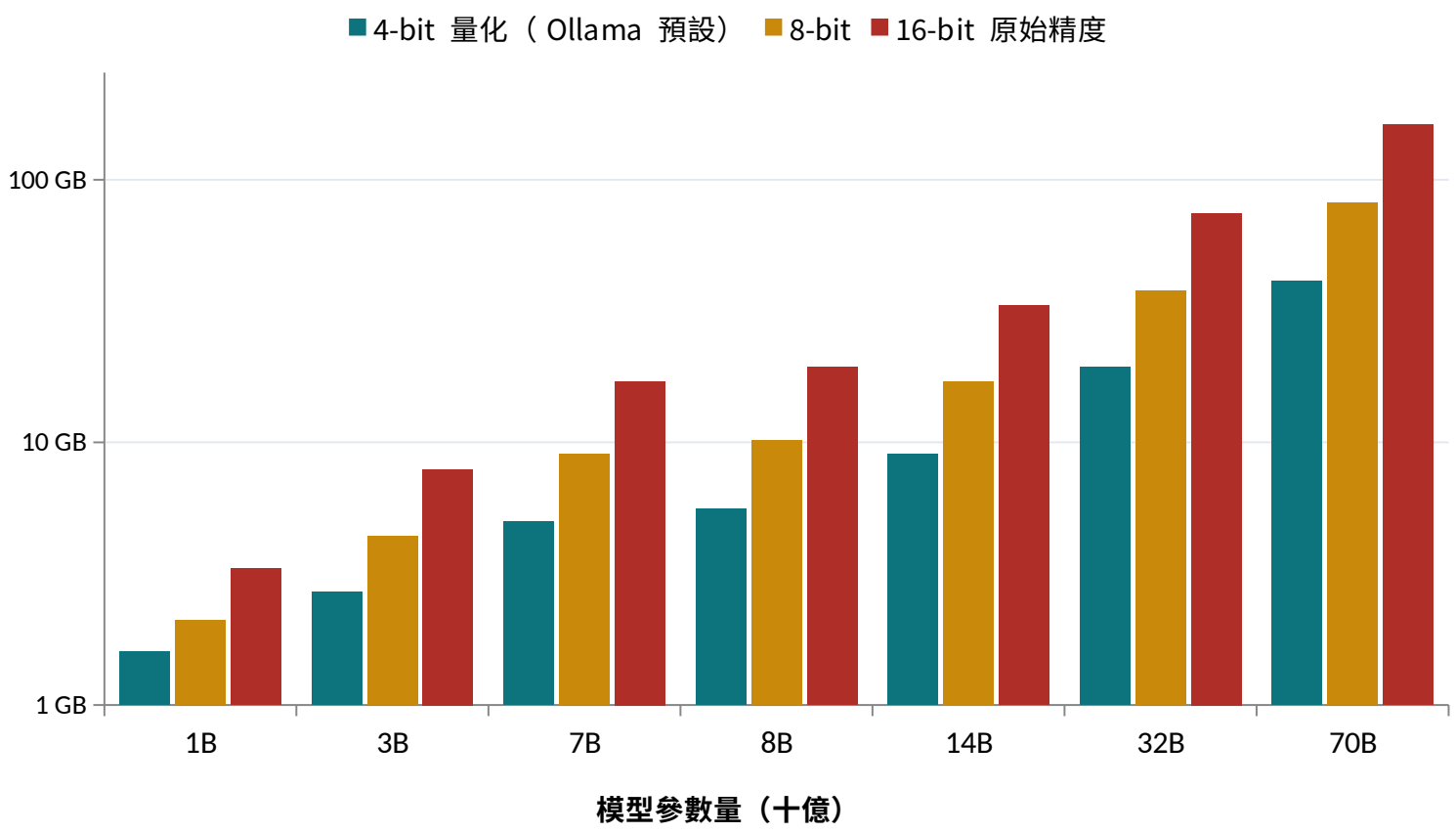
- 格式轉換：把散文改寫成 SOAP 或條列
- 摘要既有文本：內容都在輸入裡，不需要它「知道」什麼
- 分類與抽取：判斷情緒、抽出關鍵欄位
- ★ 搭配 RAG 時表現會明顯提升——事實由知識庫提供

本地 8B 模型做不好的

- 需要廣泛世界知識的問答
- 多步推理（例如複雜的給付資格判斷）
- 長文的一致性維持
- ★ 繁體中文醫療術語：多數開源模型的訓練語料以簡體為主

★ 實務折衷：敏感資料的前處理（去識別化、抽取）用本地模型，去識別化後的推理再用雲端。

實例：我的電腦跑得動多大的模型？



16 GB 筆電

可用記憶體約 12 GB：4-bit 下 8B（約 5.6 GB）、14B（約 9.0 GB）都跑得動。

8B 模型的三種精度

4-bit 約 5.6 GB、8-bit 約 10.2 GB、16-bit 約 19.4 GB。量化省下大量記憶體，能力只小幅下降。

70B 要多少

4-bit 也要約 41.2 GB——一般筆電跑不動，這就是雲端 API 存在的理由。

估算：參數量 × 位元數 ÷ 8 × 1.15（執行時的額外開銷）＋ 1 GB；縱軸為對數。經驗法則：所需記憶體不超過電腦記憶體的 75%。有獨立顯示卡時看的是顯示卡記憶體（VRAM）。

PART 09

Hugging Face 與課堂實作

Model Hub & Hands-on

模型從哪裡來？以及在 AI Studio 裡實際跑一次生成式模型。

Hugging Face： 模型的集散地

它是什麼

- 超過百萬個開源模型的公開平台，可下載權重、看評測、試玩
- 同時提供 datasets （資料集）與 spaces （線上 Demo ）
- ★ 學術與實務上「找模型」的第一站

怎麼挑一個模型

- 看任務標籤： text-generation 、 text-classification 、 sentence-similarity 等
- 看模型卡（ Model Card ）： 訓練資料、預期用途、已知限制、授權條款
- ★ 授權條款一定要看：有些禁止商用，有些要求標註來源

量化版本（ GGUF ）是什麼

- 把權重的精度降低（如 16 位元 → 4 位元），檔案變小、記憶體需求降低
- 代價是能力略降，但多數任務感受不明顯

▸ Ollama 下載的就是量化版本——這是它能在筆電上跑的原因

原講義： GAI 1. Introduction to Generative AI.pptx　pp.57–59 （ TheBloke ／ QuantFactory 模型庫、 llama.cpp ）

課堂實作｜AI Studio 的生成式模型範例

★ AI Studio Repository / Generative Model （需安裝 Generative Models 擴充）

開啟檔案

```
Generative Model/
  1 Generating Text with GPT2.rmp
  2 Sentiment analysis .rmp
  Basic LLMs.rmp
  Generative Models -T5 Base.rmp
  Finetune Text2Text Generation.rmp
  RAG - restaurant suggestion.rmp
  RAG-restaurant new data.csv
```

預期結果

- 溫度越高，同一提示詞的輸出差異越大
- T5 適合轉換任務， GPT2 適合續寫
- RAG 範例能引用到 csv 裡的實際資料
- 無 RAG 時模型會「編」出不存在的餐廳

操作步驟

1. 開啟「 Basic LLMs.rmp 」，觀察它接的是哪個 Hugging Face 模型
2. 執行「 1 Generating Text with GPT2 」，改變 temperature 各跑三次
3. 比較同一個提示詞在不同溫度下的輸出差異
4. 開啟「 Generative Models -T5 Base 」，注意這是 Encoder-Decoder 架構
5. ★ 執行「 RAG - restaurant suggestion 」——這是 M8 的預告
6. 在 RAG 範例中把 new data.csv 換成自己的資料，觀察召回結果變化
7. 記錄：同一個問題，有無 RAG 的回答差在哪

本模組的 AI Studio 範例對照

範例檔	示範什麼	對應本講義
Basic LLMs.rmp	接 Hugging Face 模型、基本推論	PART 7 、 PART 9
1 Generating Text with GPT2.rmp	Decoder-only 的續寫行為、 temperature 效果	PART 3 、 PART 6
2 Sentiment analysis .rmp	用預訓練模型做分類（ Encoder-only 的典型用法）	PART 3
Generative Models -T5 Base.rmp	Encoder-Decoder 的序列轉換	PART 3
Finetune Text2Text Generation.rmp	微調實際長什麼樣（觀摩，不要求完成）	PART 5
RAG - restaurant suggestion.rmp	★ RAG 的最小可運行範例， M8 的預告	PART 4 、銜接 M8

PART 10

提示詞基礎

Prompting Fundamentals

M8 會把提示詞系統化成四段式結構。

這一節先建立最基本的直覺：為什麼「怎麼問」會差這麼多。

提示詞的四個層次

層次 1　直接提問　「什麼是高血壓？」

得到什麼：一段通用百科式說明。長度不定、格式不定、深淺不定。

問題：每次問結果都不一樣，無法用在系統裡。

不能放進系統

層次 2　＋角色與對象　「你是衛教護理師，請向 70 歲長者說明高血壓」

得到什麼：語氣與用詞明顯調整，開始像個人在講話。

進步：可控性提高。但格式仍不穩定。

還是不夠穩

層次 3　＋結構與限制　「…分三點說明，每點 50 字內，不提藥物名稱」

得到什麼：格式穩定、長度可預期、危險內容被擋掉。

進步：★ 這是「可以放進系統」與「不行」的分界線。

★ 可以放進系統

層次 4　＋範例與替代動作　「…格式如下範例；不確定時回答『建議洽詢醫師』」

得到什麼：格式高度一致，且知識邊界外有明確行為。

★ 這就是 M8 的四段式 System Prompt。

★ 可以交付

★ 多數人停在層次 1–2，然後抱怨「AI 不好用」——差別不在模型，在你給了多少約束

層次 1　直接提問

得到一段通用說明。長度、格式、深淺都不受控。

層次 2　角色與對象

語氣與用詞開始調整，像個人在講話了。

★ 層次 3　結構與限制

格式穩定、長度可預期、危險內容被擋掉。
這是「可以放進系統」與「不行」的分界線。

層次 4　範例與替代動作

格式高度一致，知識邊界外有明確行為。
這就是 M8 的四段式 System Prompt。

★ 原生圖形，可直接在 PowerPoint 編輯

同一個需求，四種問法的實際輸出對照

需求：向 70 歲高血壓長者說明飲食注意事項

實際輸出對照（節錄）

【層次 1】「什麼是高血壓的飲食注意事項？」
→ 高血壓患者應注意鈉的攝取，建議每日鈉攝取量低於 2300 毫克…（續 400 字）
問題：太長、有數字（來源不明）、用詞偏學術、每次結果都不一樣

【層次 2】「你是衛教護理師，請向 70 歲的高血壓長者說明飲食注意事項」
→ 阿公阿嬤您好，我來跟您說說吃東西要注意的地方…（續 300 字）
進步：語氣親切、用詞淺白 ｜ 仍有問題：長度不定、沒有結構

【層次 3】＋「分三點，每點 50 字以內，不提任何藥物名稱」
→ 1. 少放鹽：煮菜時鹽巴減半，可以用蔥薑蒜提味。
2. 少吃醃漬品：醬瓜、豆腐乳、香腸這類要盡量避免。
3. 多吃蔬果：每餐半碗青菜，水果一天兩份。
★ 進步：格式穩定、長度可控、危險內容被擋掉 → 可以放進系統了

【層次 4】＋「格式如上範例；若問題超出飲食範圍，回答『這部分建議詢問您的主治醫師』；結尾固定加上免責聲明」
→ 格式與層次 3 相同，但問到「我可以停藥嗎」時會正確拒答，且每則結尾都有「本說明僅供參考，請依醫囑調整」
★ 進步：知識邊界外有明確行為 → 這才是可以交付的東西

三個立刻可用的提示詞原則

① 給替代動作，不要只給禁令

- ✗ 「不要編造」——模型不知道「不編造」的時候該做什麼
- ✓ 「知識庫沒有的內容，回答『查無相關資料，建議洽詢○○』」
- ★ 這是 M8 抑制幻覺最有效的單一手法

② 範例比規則有效

- 寫十條規則，不如給兩個「輸入 → 標準輸出」的完整示範
- 模型會模仿範例的結構、語氣、詳盡程度——連你沒說的部分都會模仿
- 醫療場域可直接用 SOAP 或 ISBAR 當範例格式

③ 把「要什麼」寫清楚，比把「不要什麼」寫清楚更重要

- 明確指定：幾點、每點多少字、用什麼稱謂、要不要標來源
- 模糊的要求會得到模糊的結果，然後你會以為是模型不好
- ★ 一個好的提示詞，讀起來應該像一份給新人的工作指示

M8 W13 會把這些原則組織成可重複使用的四段式結構：角色 → 步驟 → 限制 → 格式。

上下文工程：不只是把 Prompt 寫好

模型單次能看到的內容就這麼大。裡面放什麼、放多少，是你要設計的。



↑ 一個典型 RAG 應用的上下文組成（總計約 5,000 tokens）

- System Prompt 450 tk
- 知識庫召回段落 1500 tk
- 對話歷史 600 tk
- 工具回傳結果 450 tk
- 使用者提問 30 tk
- （剩餘空間） 1970 tk

「中間迷失」：正中間的資訊最容易被忽略



Top-K 從 5 調到 10：召回 token 翻倍，品質未必變好



★ 做法：先召回 20 段 → Rerank 精排 → 只取前 5 段送進去

對話歷史會無限成長
超過上限就會被截斷 —— 通常最早的內容先被丟掉

關鍵資訊不能被截掉
過敏、禁忌等資訊要固定寫在 System Prompt 裡

★ 重要的東西放頭尾
System Prompt 放最前、使用者提問放最後

★ 監測實際用量
Dify 的 Tracing 會顯示每次送出的 token 組成

★ 業界近年把這件事叫「上下文工程」（Context Engineering）

比「提示詞工程」更貼近實務要解的問題：這 5,000 個 token 該怎麼分配。

上下文是有限資源

System Prompt、召回段落、對話歷史、工具結果
都在搶同一個空間。

★ 召回段落佔最大宗

Top-K 從 5 調到 10，召回段落翻倍、總輸入約增六成，品質未必變好。

★ 「中間迷失」現象

放在上下文正中間的資訊，模型最容易忽略（Liu et al., 2023）。
重要的東西放頭尾。

為什麼這個詞更貼切

「提示詞工程」聽起來像在雕字句；
實務上真正要解的是「這 5000 個 token 該怎麼分配」。

★ 原生圖形，可直接在 PowerPoint 編輯

上下文管理的四個實務手法（M8 會實際設定）

① 對話歷史要有策略地截斷

- 最簡單：只保留最近 N 輪，但早期的重要資訊會消失
- 較好：最近 N 輪 + 一段摘要壓縮更早的內容
- ★ 醫療注意：過敏、禁忌等關鍵資訊不能被截掉

② 召回段落要精不要多

- 先召回 20 段 → Rerank 精排 → 只取前 5 段送進去
- 比直接取 20 段便宜、也比直接取 5 段準確

③ 把可以離線做的事移出上下文

- 查表、算術、格式驗證交給程式碼節點，不要讓 LLM 讀一大張表
- ★ 這又回到那條原則： LLM 做判斷，程式做計算

④ 監測實際用量，不要憑感覺

- Dify 的 Tracing 會顯示每次實際送出的 token 數與組成
- 上線後定期看： context 有沒有悄悄變胖、成本有沒有偏離預估

PART 11

界線與銜接

Ethics & Bridge to M8

技術能做到的事，不等於你應該做的事。
尤其在醫療與長照場域。

生成式 AI 在醫療場域的四條紅線

紅線	為什麼	課程要求
不得提供診斷	診斷是醫師的法定職權， AI 給診斷可能構成密醫行為	System Prompt 明文禁止；偵測到求診斷時導向就醫建議
不得提供處方與劑量	錯一個數字後果不可逆；且無專業人員把關	明文禁止藥物名稱與劑量建議，導向藥師或醫師
不得使用真實個案資料	個資法與機構倫理規範；一旦送進雲端無法收回	★ 一律使用去識別化或公開資料集，即使你有調閱權限
輸出必須標示 AI 生成與免責	使用者有權知道這是機器產生的內容	所有輸出結尾固定加上「僅供參考，最終應由專業人員判斷」

★ 這四條在期末專題的評分中屬於「不符即扣分」項目， M9 會再深入討論偏誤與可解釋性。

還有兩個容易忽略的問題

① 偏誤會被放大，而且不易察覺

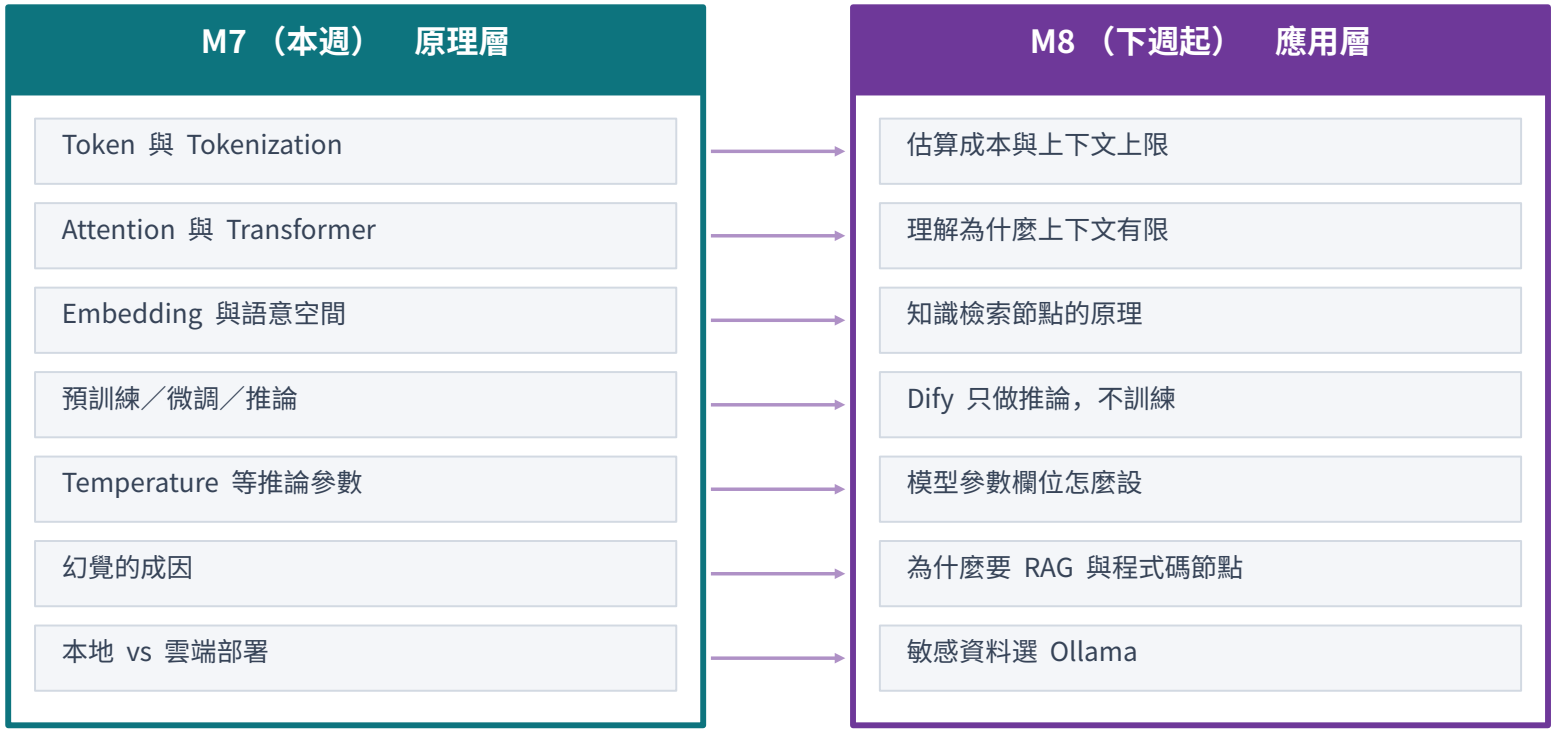
- 訓練語料以英文與西方醫療情境為主，對本地情境的適配可能不足
- 少數族群、罕見疾病、非典型表現的資料在語料中本就稀少
- ★ 模型對這些情況一樣會自信作答——這比它說「不知道」危險得多

▸ M9 會用實際案例深入討論

② 使用者會過度信任

- 流暢、有條理、語氣專業的回答，讀起來就是比較可信
- 但流暢度與正確性完全無關——這兩件事在 LLM 裡是分開的
- ★ 設計上要主動降低過度信任：標註來源、標示不確定、強制免責

M7 → M8： 原理如何變成操作



★ M7 的每一個概念，在 M8 都會變成一個你要設定的欄位或要做的判斷

這一週看起來很理論，但它決定了下三週你會不會「知其所以然」。

下週開始 M8 W13： Dify 平台入門與提示詞工程 —— 記得先把 Ollama 裝好。

左邊是這一週

看起來很理論的七個概念。

右邊是下三週

每一個概念在 Dify 裡都會變成一個你要設定的欄位，
或一個你要做的判斷。

★ 為什麼順序是這樣

先學工具再學原理，你會變成「照抄設定值的人」；
先學原理，你才知道為什麼要那樣設。

下週開始

M8 W13： Dify 平台入門與提示詞工程。
記得先把 Ollama 裝好。

★ 原生圖形，可直接在 PowerPoint 編輯

Take-Home Assignment ｜ M7 生成式 AI 基礎

W13 上課前繳交 ・ 不另計分（隨堂 Lab）

作業說明

【任務一】建立你自己的模型評測基準 （40%）
1. 針對你打算做的期末專題領域，寫出 10 個具代表性的問題
（例：長照給付、衛教問答、症狀分級——不要用真實個案）
2. 選 2 個模型：一個雲端（免費額度即可）、一個本地（Ollama）
3. 同樣的 10 個問題各問一次，記錄答案
4. 逐題標註：事實正確 ✔ / 部分正確 △ / 錯誤或編造 ✘
5. 統計兩個模型的分布，寫 200 字比較

【任務二】幻覺觀察 （30%）
1. 設計 3 個「知識庫外」的問題（模型應該不知道答案的）
例：貴院某科的門診時間、某個虛構的給付項目
2. 觀察模型的反應：它會說不知道，還是會編?
3. 同一個問題問 5 次，記錄答案是否一致
4. 寫 150 字說明你觀察到的幻覺樣態

【任務三】Ollama 本地部署 （30%）
1. 安裝 Ollama 並下載至少一個模型
2. 截圖：ollama list 的輸出 + 一次成功的對話
3. 寫 100 字：本地模型與雲端模型，你感受到的差異

【繳交】PDF 一份，含截圖與表格。檔名：M7_學號_姓名.pdf
【提醒】★ 不得使用任何真實病歷或個案資料，違者本次作業不計分。

任務一： 10 題評測表（雲端 vs 本地）

#	問題（虛構，不用真實個案）	雲端模型 ✔ △ ✘	本地模型（Ollama） ✔ △ ✘
1	例：失能等級第 4 級可以申請哪些服務?		
2			
3			
4			
5			
6			
7			
8			
9			
10			

同一組 10 題分別問雲端與本地模型，逐題標 ✔（正確）、△（部分正確）、✘（錯誤或編造）。統計兩邊的 ✔ 數，寫下落差出現在哪類問題。網站版可直接填寫並下載 CSV。

任務二：幻覺觀察紀錄（3 題 × 5 次）

問題	第 1 次	第 2 次	第 3 次	第 4 次	第 5 次
例：本院心臟科週三下午門診時間?					
例：「長照 3.5 樂齡加給」怎麼申請？（虛構項目）					
例：某虛構藥品的建議劑量?					

同一題問 5 次、記下每次的回答。觀察：答案是否每次都不一樣？有沒有說「不知道」？虛構的項目它有沒有照樣編出申請流程？這就是幻覺的實際樣子。

本模組重點回顧

1. 生成式模型學的是「資料長什麼樣」，所以「編」是它的正常運作，不是故障
2. Token 決定你的帳單、上下文上限，以及它為什麼數不清字數
3. Attention 讓模型能連上遠距離的詞，代價是計算量隨長度平方成長
4. RAG 用 Encoder-only 做嵌入、 Decoder-only 做生成——這是兩件事
5. ★ 讓模型懂你的領域：先試提示詞 → 不夠加 RAG → 最後才考慮微調
6. Temperature 調低能穩定用詞，但擋不住內容編造——那要靠 RAG

M7 名詞速查表　生成式 AI 基礎

（1 / 5）

中文	English	一句話說明
生成式模型	Generative Model	學習資料分布 P(x)，能產生新內容；相對於判別式模型
判別式模型	Discriminative Model	學習分界 P(y|x)，回答「屬於哪一類」； M2–M5 的分類與迴歸都是這類
GAN	Generative Adversarial Network	生成器與鑑別器對抗學習；用於影像合成與合成資料
VAE	Variational Autoencoder	編碼到潛在空間再解碼；用於異常偵測與降維
擴散模型	Diffusion Model	從雜訊逐步去噪生成；目前影像生成主流
Transformer	Transformer	基於注意力機制的架構；所有主流 LLM 的底層
注意力機制	Attention Mechanism	對序列中每個位置計算關聯權重，不受距離衰減影響
自注意力	Self-Attention	序列對自己做注意力計算，捕捉內部的長距離關係
多頭注意力	Multi-Head Attention	並行多組注意力，可同時追蹤語法、語意等不同關係

M7 名詞速查表　生成式 AI 基礎

（2 / 5）

中文	English	一句話說明
Token	Token	模型處理文字的最小單位；中文約一字一 token，英文約 0.75 詞
斷詞／分詞	Tokenization	把文字切成 token 的過程；各模型的切法不同
上下文長度	Context Window	單次能處理的 token 上限，含提示詞、召回段落與對話歷史
嵌入	Embedding	把文字轉成數值向量，語意相近者向量距離相近
餘弦相似度	Cosine Similarity	以向量夾角衡量相似度， RAG 召回排序的常用指標
向量資料庫	Vector Database	儲存嵌入向量並支援最近鄰搜尋的資料庫
Encoder-only	Encoder-only Architecture	雙向理解型架構（BERT 系）；用於嵌入與分類
Decoder-only	Decoder-only Architecture	單向生成型架構（GPT 系）；所有主流對話 LLM
Encoder-Decoder	Encoder-Decoder Architecture	序列轉序列架構（T5 、 BART ）；用於翻譯與摘要

M7 名詞速查表　生成式 AI 基礎

（3 / 5）

中文	English	一句話說明
預訓練	Pretraining	以海量無標註語料訓練基礎模型；成本極高
微調	Fine-tuning	在既有模型上以領域資料再訓練，調整權重
推論	Inference	呼叫已訓練好的模型取得輸出；權重不變，按 token 計費
監督式微調	Supervised Fine-tuning, SFT	以標註的輸入輸出對進行微調
人類回饋強化學習	RLHF	以人類偏好排序優化模型輸出
自監督學習	Self-Supervised Learning	從無標註資料自動產生訓練目標（如遮字預測）
領域適配	Domain Adaptation	用領域語料讓模型熟悉該領域用語與行文
參數量	Parameters	模型權重的數量，如 7B 、 70B ；大致對應能力上限
混合專家	Mixture of Experts, MoE	僅活化部分參數以提升推理效率的架構

M7 名詞速查表　生成式 AI 基礎

（4 / 5）

中文	English	一句話說明
多模態	Multimodal	能處理文字以外的輸入，如影像、音訊
Temperature	Temperature	控制取樣隨機性；低溫可重現，高溫多樣
核採樣	Top-p / Nucleus Sampling	只從累積機率前 p 的候選中取樣
最大輸出長度	Max Tokens	限制單次輸出的 token 數
幻覺	Hallucination	產生流暢但不正確的內容；為機率生成機制的必然產物
Hugging Face	Hugging Face	開源模型與資料集的集散平台
模型卡	Model Card	模型的說明文件，載明訓練資料、用途、限制與授權
量化	Quantization	降低權重精度以縮小模型體積與記憶體需求
GGUF	GGUF	量化模型的常見檔案格式， Ollama 使用此格式

M7 名詞速查表　生成式 AI 基礎

（5 / 5）

中文	English	一句話說明
Ollama	Ollama	在本機執行開源 LLM 的工具，資料不離開電腦
AGI	Artificial General Intelligence	通用人工智慧；具人類水準的跨領域認知能力，目前尚未達成